Grado en Ciencia e Ingeniería de Datos

PROCESAMIENTO DEL LENUAJE NATURAL

Práctica 2: Clasificación de texto para análisis de sentimiento

Comienzo	martes 20 de octubre de 2026
Finalización	martes 17 de noviembre de 2026 (hasta las 15:50h)

OBJETIVOS

Esta segunda práctica trata la clasificación automática de texto orientada al análisis de sentimiento, abordando dos objetivos principales:

- Aprender a extraer características lingüísticas para generar representaciones vectoriales dispersas de textos. Las representaciones vectoriales generadas no serán densas, es decir, no serán embeddings numéricos preentrenados como Word2Vec o GloVe.
- Aprender a construir y evaluar modelos de clasificación supervisada para la detección de la polaridad de opinión en reseñas de usuario.

Para ello, de forma gradual, se codificará en Python una serie de scripts que nos permitirán hacer un flujo de trabajo reutilizable, desde la extracción de los textos hasta el entrenamiento y evaluación de modelos de clasificación. Durante esta práctica se hará uso de las bibliotecas NLTK y Scikit-learn.

El corpus de reseñas preprocesadas y el script reader resultantes de la Práctica 1 serán el material de trabajo fundamental en esta práctica. Si es necesario, el corpus se puede modificar o extender en esta práctica, tanto aumentando el número de reseñas como el número de operaciones. Además, incluiremos más técnicas de preprocesamiento de texto que podrán ser activadas o desactivadas en cada ejecución. Estas operaciones serán:

- Análisis gramatical (PoS tagging) de palabras.
- Análisis sintáctico de dependencias de oraciones.
- Eliminación de stopwords.
- Stemming o lemmatization.

CASO DE USO

En la Práctica 1 se construyó un corpus anotado de reseñas en inglés sobre productos de Amazon, que, al menos, incluye los textos de las valoraciones y su rating o puntuación numérica (entre 1 y 5). Además, se aplicó la API de NLTK para la creación de corpus categorizados, diferenciando tres clases o etiquetas: “positiva”, “neutra” y “negativa”. Tanto en la práctica anterior como en esta se deja a libre elección las etiquetas a utilizar y los umbrales de ratings que indican que reviews van a cada categoría.

Es importante tener en cuenta el posible desbalanceo del conjunto de datos etiquetado y tomar las medidas que se consideren oportunas para garantizar el correcto funcionamiento de los modelos de clasificación"

ESTRUCTURA

La práctica consta de los siguientes ejercicios a realizar en el orden establecido:

1. Actualización de la clase Reader para incluir la extracción de características lingüísticas de las reseñas.
2. Implementación de la clase que cargue el corpus para su clasificación
3. Diseñar e implementar el gestor del corpus.
4. Preprocesamiento básico del corpus y predicción sobre el corpus final.

Para esta práctica, se puede descargar el fichero scr.zip, que contiene el esqueleto de todos los scripts que se deben implementar en los ejercicios.

A continuación, se proporciona el enunciado de cada uno de estos ejercicios.

EJERCICIO 1: Extracción de características lingüísticas de reviews

Se pide incluir en la plantilla de código fuente que se proporciona, los scripts desarrollados en la Práctica 1, tanto para la descarga y extracción del corpus como el script que contiene la clase Reader que se ha implementado para acceder a los textos de las reviews.

A continuación, se pide implementar una nueva función dentro de la clase Reader. La función será `tokenized`, y devolverá los textos incluyendo sus etiquetas gramaticales (*PoS tagging*). Al igual que pasa con el resto de las funciones del Reader que devuelven textos, esta función debe tener como entrada dos parámetros, uno para devolver el resultado solo para uno o varios textos concretos (fileid), y otro para devolver el resultado de una o varias categorías.

EJERCICIO 2: Implementación de la clase para el cross-validation

Se pide completar el desarrollo del script `corpus_loader.py`. Este script tiene como objetivo preparar el corpus para poder validarlo mediante *cross-validation.* En concreto, su funcionalidad es guardar los identificadores de las reviews y, al iterar sobre ellos, generar diferentes particiones de entrenamiento y test, definiendo N particiones.

Los métodos que hay que implementar son los siguientes

- `__init__`: crea el objeto. Su tarea consiste en guardar el objeto reader, configurar la validación cruzada que se ha elegido y obtener los identificadores de las reviews.
- `fileids`: devuelve todos los identificadores o solo los que se seleccionen por parámetro.
- `documents`: devuelve el texto de los ficheros seleccionados.
- `labels`: devuelve la categoría de cada texto seleccionado.
- `__iter__`: método que permite iterar por cada partición, devolviendo los textos de entrenamiento y de test y las etiquetas de los casos de entrenamiento y de test.

EJERCICIO 3: Clase para la normalización de los textos

A continuación, se pide desarrollar el script `text_normalizer.py`. Este script contiene la clase `TextNormalizer`. Esta clase debe funcionar como un "transformador" que normalice los textos del corpus para poder utilizarlos en el proceso de clasificación. La clase TextNormalizer tiene que funcionar con el pipeline de Scikit-learn. Por lo tanto, debe heredar de las clases `BaseEstimator` y `TransformerMinMix` de la librería Scikit-learn.

A parte de implementar los métodos necesarios de esta herencia, esta clase debe implementar los siguientes métodos:

- `_is_punct`: comprueba si el token que se pasa por parámetro está compuesto únicamente por signos de puntuación.
- `lemmatize`: lematiza el token usando la categoría gramatical que tiene asignado. Si no reconoce la etiqueta gramatical, por defecto se puede tratar como sustantivo.
- `normalize`: tokeniza y etiqueta gramaticalmente el texto. Además, omite las puntuaciones y devuelve todos los tokens del texto lematizados y en minúsculas.

EJERCICIO 4: Implementar clasificador usando el pipeline de Scikit-learn

En este ejercicio, se pide implementar el script `sentiment_classifier.py`. Aquí se encuentra la clase `SentimentClassifier` que contendrá todo el pipeline para poder hacer una ejecución completa del proceso de clasificación de las reviews.

Esta clase definirá los pasos a ejecutar en un pipeline, que serán:

1. Normalizar los textos.
2. Vectorizarlos (por ejemplo, aplicar TF-IDF).
3. Reducir la dimensionalidad de los vectores, si se incluye en la configuración.
4. Añadir el modelo clasificador incluido por el usuario.

Una vez implementada la clase, se ejecutarán varios pipelines con diferentes configuraciones: diferentes modelos de clasificación, reducciones de dimensionalidad, rango de n-gramas, etc. Para ello usaremos la clase `GridSearchCV`, de Scikit-learn. Por tanto, para que esto pueda funcionar correctamente, la clase `SentimentClassifier` debe heredar de dos clases: `BaseEstimator` y `ClassifierMixin`.

Se debe implementar los métodos necesarios para la clase `SentimentClassifier`.

EJERCICIO 5: Implementar el script que lance todas las pruebas

Por último, se pide implementar el script `launcher.py`. Este script tiene como único objetivo ejecutar todas las configuraciones de clasificación de sentimientos de las reviews usando la clase `GridSearchCV` de Scikit-learn. Los pasos que se deben ejecutar en este script son:

1. Crear el objeto `CorpusReader`.
2. Crear el objeto `CorpusLoader`.

3. Por cada Fold que se haya creado, crear un objeto `GridSearchCV` donde:
    a. Se incluya el `SentimentClassifier` implementado en el ejercicio anterior.
    b. Se haga una estimación con los métodos: `LogisticRegression`, `MultinomialNB` y `SGDClassifier`.
    c. Aplicar varias configuraciones de reducciones, y sin reducción.
    d. Definir el rango de n-gramas que se probarán.
4. Ejecutar cada una de las pruebas.
5. Devolver los resultados de cada una de las configuraciones.

INFORME TÉCNICO

Elaborar un informe conciso `pln_p2_XXXX_YY_informe.pdf` que:

- De manera opcional, describa brevemente aspectos relevantes relacionados con el preprocesamiento de texto, y con la construcción y tratamiento de los conjuntos de datos.
- Describa de forma concisa las características lingüísticas utilizadas para construir cada representación vectorial.
- Describa los experimentos realizados para evaluar los modelos de clasificación para las diferentes representaciones vectoriales consideradas.
- Compare y analice los resultados de evaluación obtenidos. Se deberá identificar el modelo óptimo para la tarea de clasificación de polaridad.

Lo anterior se puede complementar con un análisis de la relevancia de las características donde se justifique la elección de los subconjuntos de características y se intente cuantificar el impacto de las diferentes características lingüísticas añadidas.

El informe técnico deberá incluir los nombres y apellidos de los autores.

ENTREGA

Como resultado de la práctica, cada equipo deberá entregar una carpeta comprimida en un fichero comprimido ZIP. El nombre de la carpeta y del fichero ZIP será `pln_p2_XXXX_YY.zip`, donde `XXXX` ha de sustituirse por el código del grupo de prácticas (7461 o 7462), e `YY` ha de sustituirse por el identificador de equipo: 01, 02, etc.

El fichero ZIP contendrá:

- El código Python (carpeta `src`) de los programas desarrollados en los cinco ejercicios.
- El corpus construido o parte de él para limitar el tamaño del ZIP a entregar.
- El informe técnico.

PLANIFICACIÓN SUGERIDA

Se propone la siguiente planificación de trabajo:

- **1ª semana**
    - Comenzar el ejercicio 1.
- **2ª semana**
    - Finalizar el ejercicio 1.
    - Comenzar el ejercicio 2.

- **3ª semana**
  - Finalizar el ejercicio 2.
  - Realizar el ejercicio 3.
- **4ª semana**
  - Realizar el ejercicio 4.
- **5ª semana**
  - Realizar el ejercicio 5.
  - Redactar el informe técnico.

BIBLIOGRAFÍA SUGERIDA

Para esta práctica, se pueden seguir los capítulos 4-6 del libro *Applied Text Analysis with Python*, accesible como recurso online desde la biblioteca digital de la UAM:

- Acceso al libro.